ARTCRAFT FLEET TECHNICAL BROCHURE · SERIES #3

BookCraft v1.0

Full-power Calibre library manager in pure Rust: search, inspect, edit, add, remove,



v1.0	3 Crates	7101	6-Gate
RELEASE VERSION	WORKSPACE TIERS	PURE-RUST LOC	ARTCRAFT VERIFIED

Repository: `D:\Dev\repos\bookcraft` · **Standard:** `CRAFT_BROCHURE_STANDARD.md` · **Compiled via:** `BookCraft v0.5.0`

Full-power Calibre library manager in pure Rust: search, inspect, edit, add, remove, convert, and ingest for the Storytold ArtCraft suite.

One binary, three surfaces: `bookcraft-cli <verb>` for humans and scripts, `bookcraft-cli mcp` for agents over stdio, and the `bookcraft-app` desktop shelf.

All three call `bookcraft_core::dispatch`, so they cannot disagree.

**Status: v0.5.0 - L1 core, L2 CLI/MCP (30 tools) and the L3 shelf (11 views: now with covers, in-shelf reading and multi-select bulk work), all verified (46 tests).

Democraft recipes live in `democraft-mcp`. See `HANDOVER.md`.**

Guarantees

- **Writes go through Calibre's own rules.** Reads open `metadata.db` read-only. Every mutation is direct SQL inside one IMMEDIATE transaction with a 30-second busy timeout, registering the same `title_sort` / `uuid4` functions Calibre's triggers call. `calibredb` is deliberately not used: it refuses every local operation while any Calibre program runs. A database that is not a Calibre library is rejected on open.
- **A backup is taken first.** Every mutation copies `metadata.db` (plus the full-text index) to a timestamped backup under `%LOCALAPPDATA%\BookCraft\backups` (reused while younger than 120 s); the backup path is in every mutation result. `book backup` takes one on demand.
- **Conversion output never overwrites.** `bookcraft convert` writes into the `--out_dir` you name and picks `<title>—2.<ext>` when the name is taken.
- **Removed books are recoverable.** `remove-book` moves the book directory to the trash area beside the backups; only `--permanent` deletes. The database row is always removed through Calibre's own cascade trigger.
- **Confinement is enforced when you ask for it.** Pass `--root DIR` (or `BOOKCRAFT_ROOT`) and every library, input and output path must resolve beneath it, including `..` traversal.
- **No secrets, no network.** Readly ingest takes a *saved* HTML page; the live browser path stays in `readly-mcp`.

Quickstart

```
# Build
cargo build --release -p bookcraft-mcp

# Is the converter engine present? What is in the library?
.\target\release\bookcraft-cli.exe --library "D:\Books\Calibre Library" status
.\target\release\bookcraft-cli.exe --library "D:\Books\Calibre Library" stats

# Find a book, then read its full record
.\target\release\bookcraft-cli.exe --library "D:\Books\Calibre Library" search --query "dune"
.\target\release\bookcraft-cli.exe --library "D:\Books\Calibre Library" info --id 42 --json

# Search book text when Calibre built its full-text index (reports which engine answered)
.\target\release\bookcraft-cli.exe --library "D:\Books\Calibre Library" grep --pattern
"blacksmith"

# Convert one book (needs Calibre's ebook-convert on PATH or BOOKCRAFT_EBOOK_CONVERT)
.\target\release\bookcraft-cli.exe --library "D:\Books\Calibre Library" convert --id 42 --from
EPUB --to txt --out-dir .\outbox

# Edit metadata (backup is automatic; fields: title authors tags series series_index
# publisher rating isbn languages comments pubdate identifiers)
.\target\release\bookcraft-cli.exe --library "D:\Books\Calibre Library" update --id 42 --set
title="Dune" --set tags="sci-fi, classic" --set rating=4

# Add and remove books (add --empty for a record without files)
.\target\release\bookcraft-cli.exe --library "D:\Books\Calibre Library" add-book --file .\dune.txt
--title "Dune" --authors "Frank Herbert"
.\target\release\bookcraft-cli.exe --library "D:\Books\Calibre Library" remove-book --id 42

# Turn a saved article page into Markdown
.\target\release\bookcraft-cli.exe ingest --html .\tests\fixtures\sample-issue.html --out
.\outbox\issue.md

# MCP server (stdio)
.\target\release\bookcraft-cli.exe mcp

# Desktop shelf (same core, in-process; --smoke runs the headless checks)
.\target\debug\bookcraft-app.exe --smoke
.\target\debug\bookcraft-app.exe
```

Verbs

Verb	What it does
`status`	Engine (`ebook-convert`) and optional library health
`list`	Books, newest first (`--limit` , `--offset`)
`search`	Substring match on title, author sort, authors, series, tags, descriptions
`info`	Full record: description, format files with real paths, cover
`grep`	Book text via `full-text-search.db` when present, else titles/descriptions
`stats`	Counts and format distribution
`convert`	`ebook-convert` into `--out-dir` , with the source format you choose
`ingest`	Saved HTML page -> Markdown artifact (front matter + body)
`schema`	Dump the Calibre schema (`sqlite_master`): tables, indexes, triggers
`backup`	Timestamped metadata backup outside the library (`--force` for a fresh one)
`update`	Set metadata fields on one book (`--set key=value` , repeatable)
`add-book`	Add a book file, or an empty record with `--empty`
`remove-book`	Remove one book (Recycle Bin unless `--permanent`)
`add-format`	Attach one format file to a book (replaces the format when present)
`remove-format`	Drop one format from a book; a no-op when absent
`rename`	Rename a tag/author/series/publisher (`--kind` , merges when the new name exists)
`duplicates`	Books sharing a normalised title
`series-gaps`	Series with missing volume numbers
`unread`	Books never opened
`reading-stats`	Totals, read/unread, ratings, years, top tags/authors
`export`	Library (or one search) as CSV or JSON into `--out`
`excerpt`	Paginated chapter text from EPUB, raw window from TXT
`open`	Open the best file with the system viewer
`random`	A random book, optionally filtered
`collection-save` / `collection-list` / `collection-run` / `collection-delete`	Named saved searches
`bulk-update`	Same fields on many books, all-or-nothing (`--ids 1,2,3`)
`tools`	Print the MCP tool surface as JSON
`mcp`	Run the MCP 2024-11-05 stdio server

Every verb accepts `--json` . The MCP surface exposes 30 tools (`book_status` ,

`book_list`, `book_search`, `book_info`, `book_grep`, `book_stats`, `book_schema`, `book_backup`, `book_update`, `book_add`, `book_remove`, `book_add_format`, `book_remove_format`, `book_rename`, `book_duplicates`, `book_series_gaps`, `book_unread`, `book_reading_stats`, `book_export`, `book_excerpt`, `book_open`, `book_random`, `book_collection_save`, `book_collection_list`, `book_collection_run`, `book_collection_delete`, `book_bulk_update`, `book_convert`, `book_ingest`, `book_init`) and accepts dotted ids too (`book.status`, …).

Full detail: `docs/TOOLS.md`.

Environment

Variable	Meaning
`BOOKCRAFT_LIBRARY`	Default Calibre library directory or `metadata.db` path
`BOOKCRAFT_ROOT`	Default confinement root
`BOOKCRAFT_EBOOK_CONVERT`	Explicit `ebook-convert` path (overrides PATH discovery)

If `ebook-convert` is absent, `status` reports `{"status":"down"}` and `convert` fails with an actionable error; nothing pretends to have converted.

Build and verify

```
just ci          # fmt-check + clippy -D warnings + cargo test
just run-cli
just run-ingest
```

The 6-gate ArtCraft bar (`mcp-central-docs/standards/RUST_ARTCRAFT_STANDARD.md` §2):

clippy clean with `unwrap/expect/panic/todo/unimplemented` denied, tests green, `cargo fmt

--check` `clean, MCP stdio probe byte-clean,` Cargo.lock `committed,` CHANGELOG.md` current.

Docs

- `docs/PRD.md` - model, verbs, boundaries, recipe targets
- `docs/TOOLS.md` - MCP tool reference
- `docs/UI_PARITY.md` - shelf contract and known limits
- `docs/PARITY.md` - calibre-mcp parity checklist and what is deliberately out of scope
- `INSTALL.md` - prerequisites and client config
- `AGENTS.md` - containment rules for agents working here
- `HANDOVER.md` - what was done, what is next, what was deliberately skipped

Status — does today / next week / non-goals

Does today: v0.3.0, 3-tier (core/mcp/app); 15 MCP tools + 17 CLI verbs; 34 test fns; backup-before-write, trash-not-delete; PARITY docs; cutover made (BookCraft is the craft, rust-calibre-mcp stays the read-only dashboard).

Next week: Phase B (catalog rename/merge, analytics, export); Calibre-version matrix; convert-roundtrip timing baseline.

Non-goals: no cloud sync; no living-room-server scope creep — library manager, versioned honestly.

Lineage — the Storytold craft-app pattern

Storytold ArtCraft (storytold/artcraft — "the IDE for artists") was the starting point. The 16 craft apps in the Storytold stable each ship one binary that is both the human CLI and the agent MCP server (stdio), headless-first, with `run.json` + report artifacts. That modus operandi and harness is what this repo implements; the code here is independent, the pattern is shared.

Product Requirements & Architecture Specification

Source plan: `mcp-central-docs/projects/craft-fleet-rebuild/PLAN.md` §5 #8,

corrected 2026-11-05: BookCraft is a full-power Calibre manager, not a thin subset.

Bucket: A (SQLite + CLI + files) -> Pure-Rust ArtCraft

Status: v0.3.0 L1/L2/L3 shipped with the write path; catalog analytics next.

1. Model

A **book** is one Calibre row: id, uuid, title, sort, author_sort, authors[], series, series_index, tags[], formats[], rating, pubdate, last_modified, path, cover, description, plus the real files on disk.

The same model covers EPUB/PDF prose, PDF magazines, and CBZ/CBR comics because Calibre stores all of them the same way. BookCraft does not distinguish "book" from "issue" in its verbs; the format and path tell you what it is.

2. Seam

Concern	Owner
Library storage, metadata CRUD, covers, full text	**BookCraft** (direct SQL under Calibre's rules)
Format conversion	**BookCraft** -> Calibre `ebook-convert`
Saved-page HTML -> Markdown	**BookCraft** `ingest`
Live magazine scraping, login, browser	`readly-mcp` (unchanged)
Server-side library service, accounts	not a craft (`calibre-mcp` server mode)
Calibre desktop GUI	Calibre itself (BookCraft never drives it)

Correction 2026-11-05: the original plan said "library managers are not crafts". That was the fuckup this revision fixes - BookCraft manages the library. `calibredb` is not used because it refuses every local operation while any Calibre program runs; BookCraft writes SQL itself inside Calibre's own trigger and transaction invariants instead.

3. Verbs (normative)

```
bookcraft-cli status
bookcraft-cli list   --limit N --offset N
bookcraft-cli search --query TEXT --limit N
bookcraft-cli info   --id ID
bookcraft-cli grep   --pattern TEXT --limit N
bookcraft-cli stats
bookcraft-cli schema
bookcraft-cli backup [--force]
bookcraft-cli update --id ID [--set key=value ...]
bookcraft-cli add-book --file FILE [--title T --authors A --tags T ...] | --empty
bookcraft-cli remove-book --id ID [--permanent]
bookcraft-cli add-format --id ID --file FILE
bookcraft-cli remove-format --id ID --format FMT
bookcraft-cli rename --kind tag|author|series|publisher --old OLD --new NEW
bookcraft-cli duplicates [--limit N]
bookcraft-cli series-gaps
bookcraft-cli unread [--limit N]
bookcraft-cli reading-stats
bookcraft-cli export --format csv|json --out FILE [--query TEXT]
bookcraft-cli excerpt --id ID [--format F] [--chapter N --chars N --offset N]
bookcraft-cli open --id ID [--format F]
bookcraft-cli random [--tag T --author A --format F]
bookcraft-cli collection-save --name NAME --query TEXT
bookcraft-cli collection-list
bookcraft-cli collection-run --name NAME [--limit N]
bookcraft-cli collection-delete --name NAME
bookcraft-cli bulk-update --ids "1,2,3" [--set key=value ...]
bookcraft-cli init
bookcraft-cli convert --id ID --to FORMAT [--from FORMAT] --out-dir DIR
bookcraft-cli ingest  --html FILE --out FILE [--title TEXT]
bookcraft-cli tools
bookcraft-cli mcp
```

`update` fields: title, authors, tags, series, series_index, publisher, rating (0-5), isbn, languages, comments, pubdate, identifiers. Every mutation takes a timestamped backup first and reports its path; `add-book` rejects duplicates unless asked.

4. Recipe targets (implemented in `democraft-mcp`, not here)

1. `book/search-and-info` - search a fixture library, dump one record, assert keys.
2. `book/convert-roundtrip` - EPUB -> PDF/TXT, measure bytes; CBZ -> PDF when a comic fixture is present.
3. `book/fulltext-find` - FTS query with snippet; assert the reported engine.
4. `book/readly-ingest-sim` - `tests/fixtures/sample-issue.html` -> `issue.md`, assert title and word count; never a browser.
5. `book/manage-roundtrip` - scratch library, backup, add, update, verify, remove; asserts the round-trip payloads (DONE 2026-11-05).

Fixtures must be repository-local and DRM-free. No real library path goes into a recipe.

5. Safety and failure modes

- Reads open the library read-only; writes run inside one IMMEDIATE transaction with a

30-second busy timeout, so a held Calibre library fails fast instead of half-writing.

- Every mutation takes a timestamped backup outside the library first and reports it.
- `remove-book` moves files to a trash area by default; only `--permanent` deletes.
- `add-book` rejects duplicates without writing anything unless `duplicates` / `automerge`.
- A non-Calibre `metadata.db` is rejected on open.
- `--root` confinement covers library, inputs and outputs.
- Conversion never overwrites; the destination is `<title>[-n].<ext>`.
- Missing `ebook-convert` -> `status: down` + actionable error from `convert`.
- Missing `full-text-search.db` -> `engine: metadata_db`, not an empty success.
- Missing format file on disk -> `NotFound`, not a zero-byte output.
- Readly tokens never enter this repo; live scraping is out of scope.

6. Non-goals

- A server with accounts, OPDS, or a web UI.
- Embedding a converter; Calibre remains the engine.
- Duplicating `readly-mcp`'s browser automation.

7. Open decisions

1. **L3 app: built 2026-11-05, beautified the same day.** `bookcraft-app` ships eleven views over the same in-process `dispatch`: covers render from the library's own files, the Read view pages chapters in-shelf, the Library list multi-selects into bulk actions. Warm paper-and-amber theme throughout.
2. **Comic/manga specifics:** CBZ/CBR are convertible through `ebook-convert`; a dedicated page-count/cover path is not implemented.
3. **Naming:** settled - BookCraft is the craft; `rust-calibre-mcp` stays as the richer read-only dashboard (see HANDOVER). A hard cutover waits on full parity.
4. **Phase B: DONE 2026-11-05** (v0.4.0) - renames, analytics, export, excerpts, open/random, collections, bulk update, init. See `docs/PARITY.md`.
5. **Phase C (open):** semantic search/RAG, online metadata fetch, OPDS/server mode.

Complete MCP Tool & CLI Verb Reference

Thirty tools, all routed through `bookcraft_core::dispatch`. Both `book_status` and `book.status` are accepted (strict MCP hosts reject dots, ArtCraft ids use them).

Common optional arguments on every tool: `library` (Calibre directory or `metadata.db`), `root` (confinement root). They default to `BOOKCRAFT_LIBRARY` / `BOOKCRAFT_ROOT`, and can be set once in the client config.

Tool	Arguments	Returns
`book_status`	-	`app`, `version`, `ebook_convert{status,path?}`, `library{status,root,books}`
`book_list`	`limit` (1-1000, default 20), `offset`	`count`, `books[]` newest first
`book_search`	`query` (required), `limit`	`query`, `count`, `books[]`
`book_info`	`id` (required)	`book` with `description`, `format_files[{format,size_bytes,file,path,` `cover_path`, `uuid`, `sort`, `author_sort`, `last_`
`book_grep`	`pattern` (required), `limit`	`engine` (`full_text_search_db` or `metadata_db` `hits[]`
`book_stats`	-	`books`, `authors`, `tags`, `series`, `publishers`, cover/description counts, `formats[]`
`book_schema`	-	`count`, `objects[]` with `kind`, `name`, `sql` from `sqlite_master`
`book_backup`	`force`	`backup` path (fresh copy, or the recent one reused)
`book_update`	`id`, `fields` object (required)	`book_id`, `updated_fields[]`, `backup`
`book_add`	`file` (or `empty: true`), `title`, `authors`, `tags`, `series`, `series_index`, `languages`, `isbn`, `duplicates`, `automerge`	`book_id`, `new`, `format?`, `file?`, `bytes?`, `merg` `backup`
`book_remove`	`id` (required), `permanent`	`book_id`, `title`, `permanent`, `files`, `backup`
`book_add_format`	`id`, `file` (required)	`book_id`, `format`, `new`, `file`, `bytes`, `backup`
`book_remove_format`	`id`, `format` (required)	`book_id`, `format`, `removed`
`book_rename`	`kind` (`tag` / `author` / `series` / `publisher`), `old`, `new` (required)	`kind`, `from`, `to`, `merged`, `books`, `backup`
`book_duplicates`	`limit`	`groups[]` with `title`, `count`, `books[]`
`book_series_gaps`	-	`series[]` with `have`, `missing`, `volumes[]`
`book_unread`	`limit`	`count`, `books[]`, `heuristic`
`book_reading_stats`	-	`books`, `read`, `unread`, `ratings[]`, `added_by_y` `top_tags[]`, `top_authors[]`
`book_export`	`format` (`csv` / `json`), `out` (required), `query`	`output`, `format`, `count`, `bytes`
`book_excerpt`	`id` (required), `format`, `chapter`, `chars`, `offset`	`book_id`, `title`, `format`, `chapters`, `chapter`, `chapter_file`, `text`, `has_more`
`book_open`	`id` (required), `format`	`book_id`, `path`, `opened`

Tool	Arguments	Returns
`book_random`	`tag`, `author`, `format`	`book` (one random record)
`book_collection_save`	`name`, `query` (required)	`name`, `query`
`book_collection_list`	-	`names[]`, `collections{}`
`book_collection_run`	`name` (required), `limit`	`name`, `query`, `count`, `books[]`
`book_collection_delete`	`name` (required)	`name`, `deleted`
`book_bulk_update`	`ids`, `fields` object (required)	`updated`, `books[]`, `backup`
`book_convert`	`id`, `to`, `out_dir` (required), `from`	`output`, `bytes`, `from`, `to`, `engine`, `elapsed_m`
`book_ingest`	`html`, `out` (required), `title`	`title`, `source`, `output`, `words`, `bytes`
`book_init`	- (uses `library`)	`library`, `created`, `backup`

Return shape

Success: a JSON object with `"success": true` and the fields above.

Failure: `{"success": false, "error": "…", "error_type": "not_found|invalid|outside|database|io"}`.

MCP wraps the payload as one `text` content item and sets `isError: true` when `success` is false. Nothing is returned as a binary blob.

Examples

```
{"name":"book_search","arguments":{"query":"hyperart","limit":3}}
{"name":"book_info","arguments":{"id":14362}}
{"name":"book_update","arguments":{"id":14362,"fields":{"tags":"sci-fi, classic","rating":4}}}
{"name":"book_add","arguments":{"file":"D:/inbox/dune.txt","title":"Dune","authors":"Frank
Herbert"}}
{"name":"book_rename","arguments":{"kind":"tag","old":"sci-fi","new":"Science Fiction"}}
{"name":"book_export","arguments":{"format":"csv","out":"D:/outbox/lib.csv"}}
{"name":"book_convert","arguments":{"id":14362,"from":"EPUB","to":"txt","out_dir":"D:/outbox"}}
{"name":"book_ingest","arguments":{"html":"tests/fixtures/sample-
issue.html","out":"outbox/issue.md"}}
```

Notes

- Mutations never go through `calibredb`: it refuses every local operation while any Calibre program runs. BookCraft writes SQL itself inside Calibre's trigger invariants (one IMMEDIATE transaction, 30-second busy timeout, `title_sort` / `uuid4` registered).
- Every mutation takes a timestamped backup outside the library first and reports it; `book_add` rejects duplicates unless `duplicates` or `automerge` is set.
- `book_remove` moves files to a trash area by default; only `permanent: true` deletes.
- `book_rename` merges when the new name exists (deduplicated re-link, then the old row goes); the affected books keep correct sorts.
- `book_excerpt` reads EPUB (paginated chapters) and TXT; PDF and comics are refused honestly. `book_open` opens where the server runs, not where the agent sits.
- `book_convert` needs Calibre. Without `ebook-convert` the call fails with `not_found`; it never fabricates an output file.
- `book_grep` reports which engine answered. A library without `full-text-search.db` answers from titles and descriptions only - that is a scope downgrade, and the tool says so.
- Library, input and output paths must be beneath `root` when a root is configured.